Backend Developer Manual & VS Code API Calling Guide

A definitive engineering handbook covering Clean Architecture backend customization, Python / FastAPI service extension, and comprehensive API execution workflows completely within Visual Studio Code.

Target Audience: Full-Stack Engineers, GIS Developers, API Testers	**Core Framework:** FastAPI + Shapely 2.0 + SQLAlchemy 2.0
Spatial Reference: NZTM2000 (EPSG:2193) & WGS84 (EPSG:4326)	**Test Suite:** 34 Automated Pytest Specifications
Zero-Config Database: Standalone SQLite spatial (water_services.db)	**VS Code Tools:** REST Client, Launch Debugger, Tasks, Simple Browser

1. VS Code Developer Workspace Setup

Visual Studio Code is the designated development environment for the Taumata Arowai GIS Platform. With preconfigured workspace configurations located in .vscode/, developers can debug, test, and invoke all system components without requiring external software (e.g. Postman or standalone terminals).

Recommended VS Code Extensions

Install these extensions from the VS Code Marketplace (Ctrl+Shift+X / Cmd+Shift+X):

- **REST Client** (humao.rest-client): Execute runnable .http test files with 1-click 'Send Request'.
- **Python & Pylance** (ms-python.python): Provides rich IntelliSense, linting, and type checking.
- **Python Debugger** (ms-python.debugpy): Enables F5 breakpoint debugging for FastAPI and Pytest.
- **SQLite Viewer** (qwtel.sqlite-viewer): Inspect water_services.db tables and geometries directly in VS Code.

One-Click F5 Debugging: .vscode/launch.json

Pressing **F5** in VS Code launches the FastAPI backend in interactive debug mode with live hot-reloading. Breakpoints placed anywhere in controllers, spatial math algorithms, or repositories pause execution cleanly.

```
// .vscode/launch.json (Included in workspace)
{
  "name": "FastAPI: Run & Debug Backend (Port 8000)",
  "type": "debugpy",
  "request": "launch",
  "module": "uvicorn",
  "args": ["backend.app.main:app", "--host", "127.0.0.1", "--port", "8000", "--reload"],
  "env": { "PYTHONPATH": "${workspaceFolder}" },
  "console": "integratedTerminal"
}
```

DEBUGGER WORKFLOW: You can switch between launching the FastAPI server, running the Pytest suite, or re-seeding the SQLite spatial database by selecting the target from the **Run & Debug** drop-down menu in the VS Code sidebar (Ctrl+Shift+D).

2. Different Ways to Edit & Extend the Backend

The backend strictly adheres to Clean Architecture. System responsibilities are separated into distinct layers. Below are the primary workflows for modifying or extending the system:

Layer	Directory Location	Primary Responsibility

API Routes	`backend/app/api/routes/`	HTTP verbs, routing, query unpacking, auth dependencies
Schemas (DTOs)	`backend/app/schemas/`	Pydantic v2 validation models, request/response contracts
Domain Services	`backend/app/services/`	Core business logic, 7-step engine, LINZ NZTM projection
Repositories	`backend/app/repositories/`	SQLAlchemy 2.0 queries, caching, data access isolation
Database Models	`backend/app/db/models/`	Declarative table definitions matching database schema

Way 1: Adding a New API Endpoint

1. Open or create a route file in backend/app/api/routes/.
2. Define the route handler with Pydantic request/response schemas and dependencies.
3. Register the router in backend/app/api/router.py so it mounts under /api/v1 and /api.

```
# backend/app/api/routes/water_quality.py
from fastapi import APIRouter, Depends
from backend.app.services.auth_service import require_official_or_admin

router = APIRouter(prefix="/water-quality", tags=["Water Quality Telemetry"])

@router.get("/compliance-summary")
def get_compliance_summary(user = Depends(require_official_or_admin)):
    return {"total_schemes": 45, "compliant_pct": 98.4, "status": "Taumata Arowai Compliant"}
```

Way 2: Modifying Spatial Logic or Projections

All coordinate projection calculations and GEOS polygon operations live in backend/app/services/spatial_service.py. The LINZ NZTM2000 Transverse Mercator formula converts GPS WGS84 decimal degrees (EPSG:4326) into metric coordinates (EPSG:2193) allowing millimeter-accurate distance buffer checks without Mercator latitude distortion.

```
# Example: Custom spatial buffer evaluation in spatial_service.py
from shapely.geometry import Point, shape

def check_point_in_zone(lat: float, lng: float, zone_polygon_geojson: dict) -> bool:
    pt = Point(lng, lat)  # [x=lng, y=lat] in WGS84
    poly = shape(zone_polygon_geojson)
    return poly.contains(pt)
```

Way 3: Updating Seed Data & Adding Network Assets

To modify mock pipelines, valves, or test user accounts, edit backend/app/db/seed.py. Execute the seeding script at any time without leaving VS Code by pressing Ctrl+Shift+P → **Tasks: Run Task** → **Re-Seed Spatial SQLite DB**.

3. Calling All Backend APIs Using VS Code ONLY

You do NOT need external software (like Postman or Insomnia) to test the platform. VS Code provides four native, highly productive mechanisms to execute and inspect the APIs:

METHOD A: REST Client Extension (.http) **The Pro Standard:** Open backend/backend_api.http in VS Code. Click the clickable Send Request text directly above any endpoint. Response headers and JSON body render instantly in a split tab. Supports dynamic variables and JWT token chaining across requests.	**METHOD B: VS Code Simple Browser (Swagger UI)** **Interactive Visual UI:** Press Ctrl+Shift+P → Type Simple Browser: Show → Enter http://127.0.0.1:8000/docs. The full interactive FastAPI Swagger documentation opens directly inside a VS Code editor tab with 'Try it out' buttons.
METHOD C: VS Code Integrated Terminal (curl) **Fast Command-Line:** Open the integrated terminal (Ctrl+~). Execute standard curl commands formatted with JSON output piped to jq or python. Works across PowerShell, zsh, and bash.	**METHOD D: Thunder Client Extension** **Sidebar GUI:** Install the Thunder Client extension. Provides a GUI collection runner directly in the VS Code Activity Bar for organizing environments and test suites.

Dynamic Token Chaining in backend_api.http

In backend/backend_api.http, tokens from authentication requests are automatically stored in variables and injected into subsequent requests using standard REST Client syntax:

```
# 1. Login as Authorized Official
# @name officialLogin
POST http://127.0.0.1:8000/api/v1/auth/login
Content-Type: application/json

{"email": "official@taumataarowai.govt.nz", "password": "OfficialWater#2026"}

###
# 2. Extract Token Automatically
@token = {{officialLogin.response.body.accessToken}}

###
# 3. Call Protected Inspection API with Injected Bearer Token
GET http://127.0.0.1:8000/api/v1/assets?category=reticulation_main
Authorization: Bearer {{token}}
```

Preconfigured Test Accounts for VS Code Testing

Role	Email Address	Password	Assigned Capabilities
Citizen	`citizen@nzwater.org.nz`	`CitizenNZ#2026`	Public schemes, grades, incident reporting intake
Official	`official@taumataarowai.govt.nz`	`OfficialWater#2026`	Unredacted CCTV, invert depth, incident investigation updates
Admin	`admin@taumataarowai.govt.nz`	`ArowaiAdmin#2026`	Full system governance, audit logs, spatial re-seeding

4. Complete API Request & Response Reference

Below is the complete reference of all 18 backend endpoints with exact verbs, URLs, payloads, and expected JSON structures.

[GET] /api/health — *System Uptime Health Probe* (RBAC: Public)

Verifies that the FastAPI process is active and responsive.

```
VS Code Request:
GET http://127.0.0.1:8000/api/health
```

```
Response JSON (200 OK / 201 Created):
{"status": "healthy", "service": "Taumata Arowai GIS Platform API", "version": "1.0.0"}
```

[GET] /api/ready — *Database & GEOS Readiness Check* (RBAC: Public)

Verifies database connectivity and Shapely GEOS spatial engine initialization.

```
VS Code Request:
GET http://127.0.0.1:8000/api/ready
```

```
Response JSON (200 OK / 201 Created):
{"status": "ready", "database": "connected", "spatial_engine": "shapely_geos"}
```

[POST] /api/v1/auth/login — *User Login & JWT Token Issuance* (RBAC: Public)

Authenticates Citizen, Official, or Admin credentials and returns a signed HS256 JWT.

```
VS Code Request:
POST http://127.0.0.1:8000/api/v1/auth/login
Content-Type: application/json

{"email": "official@taumataarowai.govt.nz", "password": "OfficialWater#2026"}
```

```
Response JSON (200 OK / 201 Created):
{"accessToken": "eyJhbGciOi...", "tokenType": "bearer", "role": "official", "user": {"email": "official@...", "role":
 "official"}}
```

[GET] /api/v1/auth/me — *Current User Profile* (RBAC: Authenticated)

Retrieves the currently authenticated user's profile, job title, and organization.

```
VS Code Request:
GET http://127.0.0.1:8000/api/v1/auth/me
Authorization: Bearer {{officialToken}}
```

```
Response JSON (200 OK / 201 Created):
{"id": "usr-official-001", "name": "Aroha Henare", "role": "official", "organization": "Taumata Arowai - National Reg
ulator"}
```

[GET] /api/v1/addresses/search?q={query}&limit;=5 — *LINZ Cadastral Address Search* (RBAC: Public)

Searches registered LINZ parcels and street addresses matching the query string.

```
VS Code Request:
GET http://127.0.0.1:8000/api/v1/addresses/search?q=Great%20North%20Road&limit;=2
```

```
Response JSON (200 OK / 201 Created):
{"query": "Great North Road", "count": 2, "results": [{"id": "PARCEL-4015-GNR", "fullAddress": "4015 Great North Road
, Kelston...", "areaM2": 812.0}]}
```

[GET] /api/v1/addresses/{address_id} — *Cadastral Parcel Full Attributes* (RBAC: Public)

Retrieves legal parcel attributes including Lot/DP numbers, title reference, zoning code, and service connections.

```
VS Code Request:
GET http://127.0.0.1:8000/api/v1/addresses/PARCEL-4015-GNR
```

```
Response JSON (200 OK / 201 Created):
{"id": "PARCEL-4015-GNR", "lotNumber": "Lot 14", "dpNumber": "DP 48291", "titleReference": "NA192/441", "areaM2": 812
.0, "services": {"waterConnected": true, "meterLocation": "Front berm"}}
```

[GET] /api/v1/addresses/{address_id}/water-supply — *Verified Water Supply for Known Address* (RBAC: Public)

Retrieves verified water supply zone and infrastructure lineage for a known registered parcel.

```
VS Code Request:
GET http://127.0.0.1:8000/api/v1/addresses/PARCEL-4015-GNR/water-supply
Authorization: Bearer {{officialToken}}
```

```
Response JSON (200 OK / 201 Created):
{"status": "VERIFIED", "queriedAddress": "4015 Great North Road", "supplyZone": {"zoneId": "AKL-SZ-14", "zoneName": "
Waitakere Dizzy"}, "matchedPipeCompkey": 2017826}
```

[POST] /api/v1/water-supply/resolve — *7-Step Authoritative Water Supply Resolution* (RBAC: Public)

Executes 7-step spatial resolution connecting address or coordinates to supply zone and infrastructure lineage.

```
VS Code Request:
POST http://127.0.0.1:8000/api/v1/water-supply/resolve
Content-Type: application/json

{"address": "4015 Great North Road, Kelston, Auckland 0602"}
```

```
Response JSON (200 OK / 201 Created):
{"status": "VERIFIED", "queriedAddress": "4015 Great North Road", "nztm2000": {"easting": 1748200.0, "northing": 5914
300.0}, "supplyZone": {"zoneId": "AKL-SZ-14", "zoneName": "Waitakere Dizzy"}, "lineage": {"sourceDam": {"name": "Wait
akere Dam"}, "treatmentPlant": {"name": "Huia WTP"}}}
```

[GET] /api/v1/zones?limit=10 — *Drinking Water Supply Zones (AklDizzy)* (RBAC: Public)

Lists authoritative Auckland drinking water supply zones with population metrics and compliance status.

```
VS Code Request:
GET http://127.0.0.1:8000/api/v1/zones?limit=2
```

```
Response JSON (200 OK / 201 Created):
[{"zoneId": "AKL-SZ-14", "zoneName": "Waitakere Dizzy", "registeredPopulation": 182400, "supplierName": "Watercare Se
rvices Limited"}]
```

[GET] /api/v1/zones/{zone_id} — *Supply Zone Detailed Spatial Geometry* (RBAC: Public)

Retrieves detailed spatial boundary MultiPolygon and demographic metrics for a specific zone.

```
VS Code Request:
GET http://127.0.0.1:8000/api/v1/zones/AKL-SZ-14
```

```
Response JSON (200 OK / 201 Created):
{"zoneId": "AKL-SZ-14", "zoneName": "Waitakere Dizzy", "registeredPopulation": 182400, "maoriPopulation": 24500, "geo
metry": {"type": "MultiPolygon", "coordinates": [...]}}
```

[GET] /api/v1/zones/{zone_id}/buffers — *Concentric Protection Buffer Corridors* (RBAC: Public)

Retrieves 50m, 100m, 200m, 500m, and 1500m risk assessment buffer tiers for spatial proximity evaluation.

```
VS Code Request:
GET http://127.0.0.1:8000/api/v1/zones/AKL-SZ-14/buffers
```

```
Response JSON (200 OK / 201 Created):
[{"tier": "CRITICAL_PROXIMITY", "distanceMeters": 50.0, "action": "Immediate contamination notification"}, {"tier": "
BUFFER_100M", "distanceMeters": 100.0}]
```

[GET] /api/v1/networks — *Drinking Water Supply Schemes* (RBAC: Public)

Lists registered municipal drinking water schemes across the Auckland region.

```
VS Code Request:
GET http://127.0.0.1:8000/api/v1/networks
```

```
Response JSON (200 OK / 201 Created):
[{"supplyId": "SUP-AKL-001", "supplyName": "Auckland Municipal Watercare Network", "supplierName": "Watercare Service
s Limited", "complianceStatus": "Rule Compliant (Grade A1)"}]
```

[GET] /api/v1/networks/{network_id} — *Water Supply Scheme Details* (RBAC: Public)

Retrieves detailed drinking water scheme metadata by unique supply identifier.

```
VS Code Request:
GET http://127.0.0.1:8000/api/v1/networks/SUP-AKL-001
```

```
Response JSON (200 OK / 201 Created):
{"supplyId": "SUP-AKL-001", "supplyName": "Auckland Municipal Watercare Network", "sourceWaterType": "Surface Water &
 River Catchment", "registeredPopulation": 1580000}
```

[GET] /api/v1/assets?category=reticulation_main — *Reticulation Asset Inspection (RBAC Redacted)* (RBAC: Public / Official)

Retrieves physical water assets. Public users receive REDACTED fields (null); Officials receive full telemetry.

```
VS Code Request:
GET http://127.0.0.1:8000/api/v1/assets?category=reticulation_main
Authorization: Bearer {{officialToken}}
```

```
Response JSON (200 OK / 201 Created):
[{"compkey": 2017820, "assetName": "Great North Road Watermain", "nominalDiameterMm": 150, "material": "Ductile Iron"
, "cctvConditionScore": 2, "invertDepthM": 1.45, "engineerNotes": "Inspected July 2024. Grade 2 minor internal lining
 wear."}]
```

[GET] /api/v1/assets/{compkey} — *Detailed Asset by Watercare COMPKEY* (RBAC: Public / Official)

Retrieves authoritative physical infrastructure asset by unique engineering COMPKEY.

```
VS Code Request:
GET http://127.0.0.1:8000/api/v1/assets/2017820
Authorization: Bearer {{officialToken}}
```

```
Response JSON (200 OK / 201 Created):
{"compkey": 2017820, "assetCategory": "reticulation_main", "nominalDiameterMm": 150, "material": "Ductile Iron", "pres
sureClass": "PN16", "cctvConditionScore": 2, "invertDepthM": 1.45}
```

[POST] /api/v1/incidents — *Citizen Water Incident Complaint Intake* (RBAC: Public)

Logs citizen contamination complaints and automatically assigns an official reference (e.g., NZ-DW-2026-4821).

```
VS Code Request:
POST http://127.0.0.1:8000/api/v1/incidents
Content-Type: application/json

{"category": "contamination_discoloration", "address": "4015 Great North Road", "description": "Brown water from tap"
, "severity": "high"}
```

```
Response JSON (200 OK / 201 Created):
{"id": "NZ-DW-2026-4821", "status": "Submitted", "createdAt": "2026-10-13T02:40:00Z", "category": "contamination_disc
oloration", "boilWaterAdvised": true}
```

[GET] /api/v1/incidents?status={status} — *List Registered Incident Complaints* (RBAC: Public)

Lists reported incident complaints with optional filtering by investigation status or zone ID.

```
VS Code Request:
GET http://127.0.0.1:8000/api/v1/incidents?status=Submitted
```

```
Response JSON (200 OK / 201 Created):
[{"id": "NZ-DW-2026-4821", "status": "Submitted", "category": "contamination_discoloration", "address": "4015 Great N
orth Road", "severity": "high"}]
```

[PATCH] /api/v1/incidents/{incident_id} — *Official Incident Status & Triage Update* (RBAC: Official / Admin)

Updates incident investigation progress, sampling orders, and compliance officer notes.

```
VS Code Request:
PATCH http://127.0.0.1:8000/api/v1/incidents/NZ-DW-2026-4821
Authorization: Bearer {{officialToken}}
Content-Type: application/json

{"status": "Under Investigation", "officerNotes": "Water sampling kit dispatched to Kelston."}
```

```
Response JSON (200 OK / 201 Created):
{"id": "NZ-DW-2026-4821", "status": "Under Investigation", "officerNotes": "Water sampling kit dispatched to Kelston.
", "boilWaterAdvised": true}
```

5. Deep-Dive: RBAC Telemetry Redaction in Action

To verify security governance in VS Code, execute GET /api/v1/assets/2017820 with and without the Authorization: Bearer {{officialToken}} header. Notice the exact field-level differences below:

Public Citizen View (Unauthenticated / Citizen)	Official Regulatory View (Authorized Official)

<pre>{
  "compkey": 2017820,
  "assetName": "Great North Road Main",
  "nominalDiameterMm": 150,
  "material": "Ductile Iron",
  "cctvConditionScore": null,
  "invertDepthM": null,
  "engineerNotes": null
}</pre>
	<pre>{
	  "compkey": 2017820,
	  "assetName": "Great North Road Main",
	  "nominalDiameterMm": 150,
	  "material": "Ductile Iron",
	  "cctvConditionScore": 2,
	  "invertDepthM": 1.45,
	  "engineerNotes": "Inspected July 2024..."
	}</pre>

6. VS Code Troubleshooting & Quick Solutions

Common developer scenarios encountered during local development and their verified resolutions:

Issue / Symptom	Root Cause	Resolution in VS Code
401 Unauthorized on protected endpoints	Missing or expired JWT bearer token in Authorization header	Run the officialLogin request in backend_api.http first to refresh {{officialToken}}.
ModuleNotFoundError: No module named 'fastapi'	VS Code is pointing to the global system Python instead of .venv	Press Ctrl+Shift+P → **Python: Select Interpreter** → Choose .venv/bin/python.
Connection Refused (port 8000)	The FastAPI backend process is not currently running	Press **F5** to launch the server, or run Task **Start FastAPI Backend** from the Command Palette.
CORS Error in Browser	Frontend port (e.g. 3000) not listed in backend CORS origins	Ports 3000, 5173, and 4173 are pre-whitelisted in backend/app/core/config.py. Restart backend if modified.
Database reset needed	Corrupted or outdated local test data in SQLite file	Run Terminal: Run Task → **Re-Seed Spatial SQLite DB** to restore clean state in 1 second.

Developer Action Checklist:
1. Open **backend/backend_api.http** in VS Code and click Send Request above any query.
2. Press **F5** to launch interactive debugging with real-time breakpoints.
3. Run **npm run dev** in terminal to launch the full Leaflet GIS client on port 3000.